Who should have won?

A leave-behind analysis asking whether a team-game event sequence looks like a win, independent of the final scoreboard.

RESEARCH QUESTION

Can a model identify which NHL team should have won from the sequence of events in the game, rather than from a cumulative expected-goals differential?

The project reframes each game as two team-row perspectives. Each row contains the same sampled event stream, but ownership, manpower situation, time, normalized rink location, and event type are encoded relative to the represented team. The target is whether that row corresponds to the actual winner.

This abstract summarizes the claim, design, and main empirical result.

0.9290
CNN held-out AUC
2018-19 test season

1,080
event predictors
180 retained events × 6 fields

12,428
team-game rows
duplicated home/away views

358
loss rows called wins
candidate xW disagreements

CORE THESIS

The event sequence contains enough signal to predict winners out of sample, but unsupervised structure does not cleanly divide winners from losers. The best xW use case is therefore not simply reproducing the result; it is ranking games where the model probability and final outcome disagree.

Evidence layer	Main read	Why it matters
EDA	UMAP k=2, silhouette 0.4995; win rates 0.5006 vs 0.4996	structure exists but is not outcome separation
XGBoost	AUC 0.8893; strong but driven by late event ownership	flat columns overread score effects
Temporal CNN	AUC 0.9290; F1 0.8293; recall 0.9079	sequence-aware model fits the research question

Source: NHL play-by-play via nhlscraper, regular seasons 2014-15 through 2018-19; project outputs in analysis_results.json.

One game becomes two event sequences

The design choice that makes xW possible is the team-relative row: one event stream, two perspectives, one winner label per row.

Engineering logic

The raw play-by-play table is too irregular to feed directly into supervised learning: games have different lengths, events are written from the arena's home/away orientation, and some rows leak the final score if goals are left as a separate event type. The pipeline makes the data fixed-width and team-relative while preserving the game narrative.

1 Filter retained hockey actions

Faceoffs, hits, shots-on-goal, giveaways, missed shots, blocked shots, goals, and takeaways are kept. Goals are recoded to shot-on-goal so the final answer is not handed to the classifier.

2 Read every game twice

Each game produces one home-perspective row and one away-perspective row. The same event list remains in both rows; ownership and situation fields change relative to the represented team.

3 Sample 180 events across the game

Even spacing keeps early, middle, and late context while reaching the assignment's >1,000-predictor requirement. It avoids a biased 'first 180 actions only' read.

4 Encode six repeated fields

For each event i: isOwner, team-relative situation, time, normalized x, normalized y, and retained event type. This creates a compact sequence vocabulary rather than a scoreboard summary.

EVENT FIELD SCHEMA

Each sampled event contributes the same six fields in the same order: owner flag, situation, game time, normalized x, normalized y, and retained event type. The model sees these as repeated local snapshots of game state.

Fixed-width representation

The sequence length is the main modeling tradeoff. Too short loses the full-game story; too long discards many games and inflates sparse predictors. At 180 retained events, almost every game survives while each team-row still exceeds one thousand event-level predictors.

180 retained events x **6** fields per event = **1,080** event-level predictors

SPLIT AND SHAPE

Split	Raw retained events	Games kept	Team rows
Train 2014-15 to 2017-18	1,247,336	4,943 / 4,961	9,886 × 1,087
Test 2018-19	324,978	1,271 / 1,271	2,542 × 1,087

TEAM-RELATIVE SITUATION CODE

The raw NHL situation order is away goalie, away skaters, home skaters, home goalie. Reordering this around aad avent owner makes home and away sequences comparable, so a home power play and an away power play enter as the same kind of information.

LEAKAGE CONTROL

Goals are not preserved as their own event type. The model may learn from shot sequence, location, timing, ownership, and manpower, but not from a direct flag that the puck entered the net.

WHY THIS IS STILL A HOCKEY READ

The row does not reduce the game to a single expected-goals total. It preserves pressure waves, rink geography, manpower context, and event ordering across the full game.

Source: NHL play-by-play via nhlscraper, regular seasons 2014-15 through 2018-19; project outputs in analysis_results.json.

The event matrix has shape, but not a winner/loser map

The unsupervised view is valuable because it shows the representation is not random noise, while also showing that outcome labels require supervised learning.

What the projection says

PCA confirms that the fixed-width event space remains high-dimensional: 673 principal components are needed to explain 90% of variance. UMAP then reveals a broad two-cluster geometry, but those clusters have almost identical win rates.

Measure	Value	Interpretation
PCA 90% variance	673 components	many weakly correlated event channels
PC1 / PC2 variance	10.41% / 2.76%	no single axis dominates
Best KMeans k	2	broad geometry is visible
Silhouette	0.4995	reasonable cluster separation

CLUSTER WIN RATES

0.5006
cluster 0
4,139 rows

0.4996
cluster 1
5,747 rows

Interpretation: the geometry is real, but it mainly separates game style or flow patterns, not winners from losers. A good xW model therefore has to learn outcome-linked order patterns rather than rely on cluster membership.

WHY UNSUPERVISED LEARNING IS IMPORTANT

A purely supervised report can look like a black box. The EDA checks whether the engineered event matrix has meaningful structure before asking it to predict winners.

WHAT IT RULES OUT

The plot does not support a simple unsupervised ‘winner cluster’ story. That prevents an overclaim and motivates the temporal CNN as the appropriate next step.

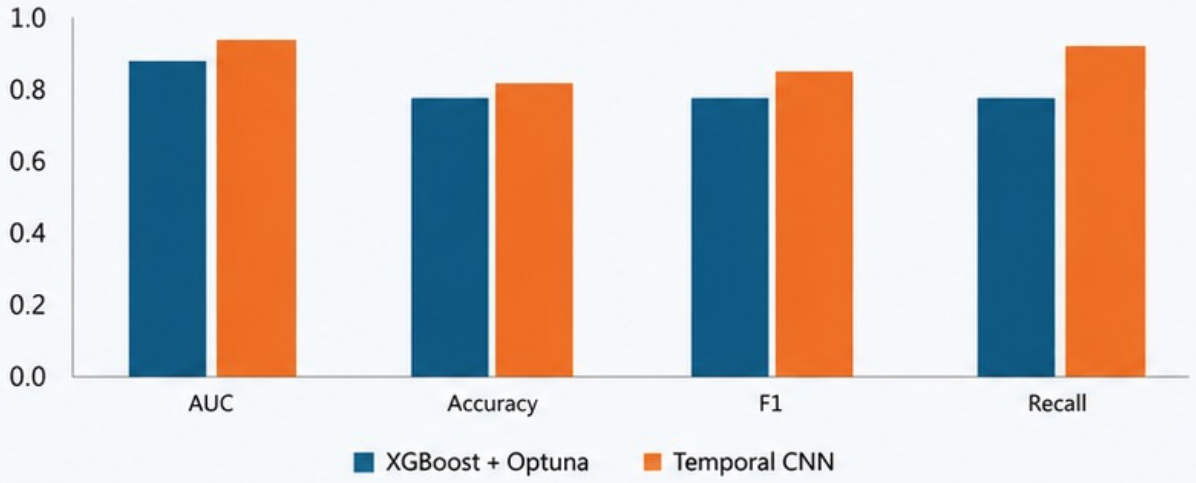
Source: NHL play-by-play via nhlscraper, regular seasons 2014-15 through 2018-19; project outputs in analysis_results.json.

The sequence model wins because order matters

XGBoost is a strong tabular baseline, but the temporal CNN better matches the question because event order is part of the evidence.

Held-out 2018-19 performance

CNN > XGBoost



Model	Accuracy	Precision	Recall	F1	AUC	Log-loss
XGBoost + Optuna	0.7714	0.7721	0.7703	0.7712	0.8893	0.4217
Temporal CNN	0.8131	0.7632	0.9079	0.8293	0.9290	0.3416

+0.0397
AUC lift
0.9290 vs 0.8893

+0.1376
recall lift
0.9079 vs 0.7703

-0.0801
log-loss change
0.3416 vs 0.4217

Interpretation, not just scores

WHAT XGBOOST LEARNED

Its highest-gain features cluster around late event positions: event_173_isOwner, event_175_isOwner, event_177_isOwner, event_178_isOwner, event_176_type, and late situation fields.

53.4%
late events owned by actual losses
last 30 event ownership average

46.6%
late events owned by actual wins
last 30 event ownership average

That is a score-effects signal. Teams that are trailing often push harder late, so a flat model can read late possession as a winner-like pattern even when it is partly desperation pressure.

WHAT THE CNN ADDS

The CNN learns local event-order patterns: ownership runs, location changes, event type, game time, and mampower situation together. It treats the 180 events as a sequence rather than 1,080 unrelated columns.

9,081
trainable parameters
small temporal model

epoch 10
best validation AUC
0.9182

Confusion matrix	Predicted loss	Predicted win
Actual loss	913	358
Actual win	117	1,154

Source: NHL play-by-play via nhlscraper, regular seasons 2014-15 through 2018-19; project outputs in analysis_results.json.

The disagreement cases are the point

A deserve-to-win measure should surface where the event-sequence probability disagrees with the final result, not merely reproduce the scoreboard.

A game-level xW read compares the two team rows

The classifier output is currently a team-row probability. For a finished xW metric, the two probabilities from the same game should be paired and converted into a disagreement score. The most interesting cases are not generic prediction errors; they are games where the losing team's event sequence looks winner-like.

Losing team row	**p(win) = 0.63**	Candidate should-have-won game: the event flow looked more like a winning performance than the final score did.
Winning team row	**p(win) = 0.37**	The model sees the opponent as having the stronger sequence case, even though the scoreboard went the other way.

INSPECTION SET

At threshold 0.5, the CNN called 358 actual loss rows wins. In a normal classifier those are false positives; in this project they are the natural candidates for should-have-won games because the model is saying the event pattern resembled a winner.

358
actual loss rows called wins
xW inspection candidates

1,154
true winning rows found
model also recovers winners

PROPOSED SCORE

For each game, pair the two rows and rank by losing-team probability minus winning-team probability. Large positive values mark the strongest 'should-have-won' claims.

Conclusions and limits

- The event sequence representation generalizes: held-out AUC 0.9290 on the unseen 2018-19 season.
- Unsupervised clusters show style/flow structure, not direct outcome separation, which keeps the claim appropriately modest.
- The temporal CNN is the better fit because it learns short event-order patterns instead of treating late-game columns independently.
- The next deliverable should be a ranked game table with actual teams, final scores, p(win) for each row, and a disagreement explanation.

Limitations: no roster or goalie quality, no explicit score state, fixed 180-event compression, and current outputs are team-row probabilities rather than a fully calibrated game-level xW score.

POTENTIAL NEXT STEP

Convert the two team-row probabilities into a game-level xW disagreement score, then rank the losing teams with the strongest event-sequence case.

Source: NHL play-by-play via nhlscraper, regular seasons 2014-15 through 2018-19; project outputs in analysis_results.json.